

Spring review 2 report

SQL Schema

```
-- USERS TABLE
CREATE TABLE Users (
  user_ID SERIAL PRIMARY KEY,
  email VARCHAR(100) UNIQUE NOT NULL,
  passwordHash TEXT NOT NULL,
  firstName VARCHAR(50),
  lastName VARCHAR(50),
  phoneNumber VARCHAR(20),
  dateOfBirth DATE,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  DietRes TEXT,
  role VARCHAR(50) NOT NULL
);

-- ANNOUNCEMENTS
CREATE TABLE Announcement (
  announcement_ID SERIAL PRIMARY KEY,
  title VARCHAR(100) NOT NULL,
  content TEXT,
  postedBy INT,
  postedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  isPublic BOOLEAN,
  FOREIGN KEY (postedBy) REFERENCES Users(user_ID)
);

-- EVENTS
CREATE TABLE Event (
  event_ID SERIAL PRIMARY KEY,
  title VARCHAR(100) NOT NULL,
  description TEXT,
  date DATE,
  location VARCHAR(100),
  image image_ID,
  maxParticipants INTEGER,
  CreatedBy INT,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  isActive BOOLEAN,
  FOREIGN KEY (image_ID) REFERENCES Image(image_ID),
  FOREIGN KEY (CreatedBy) REFERENCES Users(user_ID)
);

-- EVENT REGISTRATIONS
CREATE TABLE EventRegistration (
  registration_ID SERIAL PRIMARY KEY,
  event_ID INT NOT NULL,
```

```

    user_ID INT NOT NULL,
    registrationData TEXT,
    status VARCHAR(50),
    specialRequests TEXT,
    FOREIGN KEY (event_ID) REFERENCES Event(event_ID),
    FOREIGN KEY (user_ID) REFERENCES Users(user_ID)
);
-- NEWSLETTERS
CREATE TABLE NewsLetter (
    letter_ID SERIAL PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
    content TEXT,
    postedBy INT,
    postedAt TIMESTAMPT DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (postedBy) REFERENCES Users(user_ID)
);
-- TASKS
CREATE TABLE Task (
    task_ID SERIAL PRIMARY KEY,
    assignedTo INT NOT NULL,
    relatedEvent INT,
    dueDate DATE,
    status VARCHAR(50),
    CreatedBy INT,
    CreatedAt TIMESTAMPT DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (assignedTo) REFERENCES Users(user_ID),
    FOREIGN KEY (relatedEvent) REFERENCES Event(event_ID),
    FOREIGN KEY (CreatedBy) REFERENCES Users(user_ID)
);
-- IMAGES
CREATE TABLE Image (
    image_ID SERIAL PRIMARY KEY,
    image_Link TEXT
);

```

Explanation

The SQL schema displayed above defines the structure of the database in use for the system. It contains a table with all users, for which it stores required information such as e-mail address, password and role. Next to this there are optional fields for things like name, contact information and dietary restrictions.

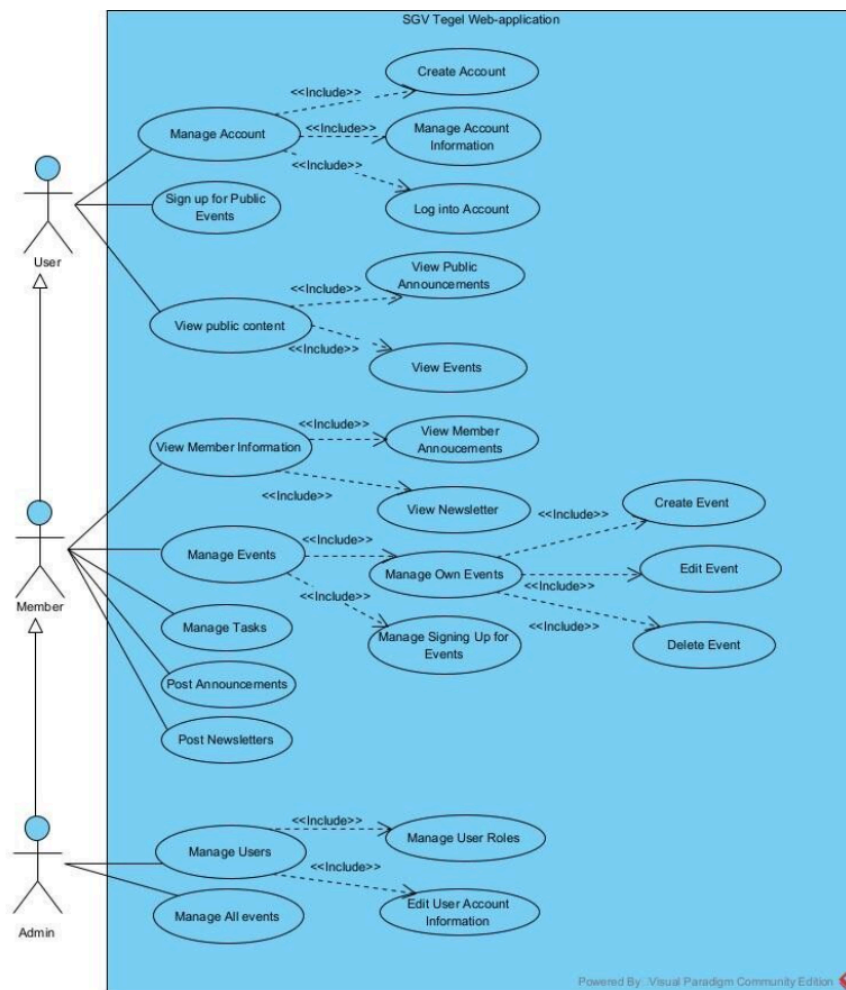
Next to this, there are a couple similar tables all storing Events, Announcements, Newsletters and Tasks respectively. These all have the similar fields ID, title and CreatedBy and CreatedAt. Additionally these tables have differing fields regarding their related uses, such as the number of maximum participants for events and for to whom a task is assigned. Furthermore, the table EventRegistration keeps track of which users signed up for what events and, if it is the case, what additional options they selected.

Finally, there is a table for storing images referenced by ID so it is possible for other tables to include image IDs instead of full images, saving a lot of space and keeping the storing of images centralized.

Progress so far

- Implemented the front end using HTML, CSS, and Bootstrap 5.
- The front end includes a responsive navigation bar, basic functionality, and interactive elements for user and event management.
- Created the initial SQL schema for users and events.

Use Case Diagram



Relationship	Description
User → Member/Admin	A user can either be a Member or an Admin, but not both.
Member → Event	A member can create events.
Admin → Announcement	An admin can make announcements.

Member/Admin → Announcement	Both can create announcements.
User → EventRegistration	A user can register for multiple events.
EventRegistration → Event	Links an event registration to a specific event.
Member/Admin → Newsletter	Both roles can create newsletters.
Member/Admin → Task	Both roles can create tasks and assign them to users.

Class Diagram



This class diagram consists of classes like users, events, announcements, newsletters, registrations, and tasks. Users are classified as either Members or Admins. Both can create announcements, newsletters, and tasks, while only Members can create events. Anybody who views the website is a user, but not all users are members, and not all members are admins. Therefore, Every Admin is a member and user, and every member is a user.

Events contain details like title, date, location, capacity, and creator. Users can register for events, with their registrations tracked by status and special requests. Announcements and newsletters store content created by Members or Admins, including timestamps and visibility settings.

Tasks are assignments linked to users and optionally to events, with due dates and statuses. The diagram defines clear relationships between entities, such as event creation, task assignment, and event registration, ensuring role-based access and activity management within the system.

To do

- Connect the front end to the backend (API/database integration).
- Implement authentication and authorisation (admin/user roles).
- Add backend logic for user management (update roles, delete users, view users).
- Add backend logic for event management (CRUD operations, chronological sorting).
- Implement feedback and error handling for admin actions.
- Conduct user testing and gather feedback.

Feedback

- Add an event request feature in the admin page.
- Events created by members should require admin approval before being published.
- Merge the personal details and signup pages for a smoother registration process.
- Create an announcements page with search functionalities for easier information access.
- Event page should have a search function incase the events page gets too crowded.